

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования

«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ И СЕТЕЙ

ОЦЕНКА

ПРЕПОДАВАТЕЛЬ

Старший преподаватель
должность, уч. степень, звание

подпись, дата

Д.А.Булгаков
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Лабораторная работа: управление вращением робота

по дисциплине: Управление виртуальным двойником

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4241в
номер группы

подпись, дата

А.А.Хусаинов
инициалы, фамилия

Студенческий билет №

2022/2783

Санкт-Петербург 2026

Цель работы

Цель работы — реализовать управление пятиосевой роборукой из веб-интерфейса через СОМ-порт, синхронизировать физическое устройство с цифровым двойником на Three.js и организовать безопасную авто-отправку изменений углов без перегрузки Serial-канала.

В ходе работы решались следующие задачи:

- описать карту сервоприводов и диапазонов;
- реализовать единый serial-протокол управления;
- связать слайдеры веб-интерфейса с 3D-моделью;
- передавать команды в Arduino через Web Serial API;
- реализовать throttled debounce для авто-отправки изменений;
- проверить работу протокола на Arduino Uno и совместимость с Arduino Nano.

Аппаратная и программная часть

Реальная установка использует Arduino Nano. Для промежуточной проверки применялась Arduino Uno без сервоприводов и экрана. Это позволило проверить Serial-протокол без риска для механики роборуки.

Используемые компоненты:

- Arduino Nano / Arduino Uno;
- 5 сервоприводов роборуки;
- веб-интерфейс на TypeScript;
- Web Serial API;
- Three.js-модель public/roboarm.glb;
- Arduino-библиотека Servo.

Карта сервоприводов

В проекте используется 5 каналов управления. Каждый канал имеет буквенный идентификатор в serial-протоколе, физический пин Arduino и допустимый диапазон.

Канал	Буква протокола	Пин Arduino	Узел робота	Диапазон
S1	A	D9	База	0..180
S2	B	D6	Плечо	0..180
S3	C	D5	Локоть	0..180
S4	D	D3	Запястье	0..180
S5	E	D11	Захват	35..90

Фрагмент конфигурации в веб-интерфейсе:

```
const SERVO_CONFIG = [  
  { id: 'A', min: 0, max: 180, initial: 90 },
```

```
{ id: 'B', min: 0, max: 180, initial: 90 },  
{ id: 'C', min: 0, max: 180, initial: 90 },  
{ id: 'D', min: 0, max: 180, initial: 90 },  
{ id: 'E', min: 35, max: 90, initial: 90 },  
] as const
```

Та же карта задана в Arduino-прошивке:

```
const byte SERVO_COUNT = 5;  
const byte servoPins[SERVO_COUNT] = {9, 6, 5, 3, 11};  
const char servoIds[SERVO_COUNT] = {'A', 'B', 'C', 'D', 'E'};  
const int servoMin[SERVO_COUNT] = {0, 0, 0, 0, 35};  
const int servoMax[SERVO_COUNT] = {180, 180, 180, 180, 90};  
int servoAngles[SERVO_COUNT] = {90, 90, 90, 90, 90};  
Servo servos[SERVO_COUNT];
```

Диапазон захвата ограничен значениями 35..90, так как механика gripper имеет меньший рабочий ход, чем остальные оси.

Протокол управления через СОМ-порт

Управление роботом выполняется текстовым пакетом:

A{angle}B{angle}C{angle}D{angle}E{angle};

Пример:

A100B120C60D90E90;

В этом пакете:

- A100 — база устанавливается на 100°;
- B120 — плечо устанавливается на 120°;
- C60 — локоть устанавливается на 60°;
- D90 — запястье устанавливается на 90°;
- E90 — захват устанавливается на 90°.

Команда завершается символом ;, по которому Arduino понимает, что пакет получен полностью.

При успешной обработке Arduino отвечает:

OK SERVOS A100B120C60D90E90;

Если пакет неполный или неверный, возвращается ошибка:

ERR invalid servo packet: A10B20

Поток данных в веб-приложении

Веб-приложение хранит текущие углы в массиве values. Слайдеры, hand tracking и другие источники управления должны обновлять именно этот массив.

При изменении слайдера выполняется несколько действий:

1. Обновляется значение в values.
2. Обновляется числовая подпись около слайдера.
3. Обновляется preview serial-пакета.
4. Вызывается setServoAngle(...) для 3D-модели.
5. Если включена авто-отправка, планируется отправка пакета в Arduino.

Фрагмент обработчика слайдера:

```
slider.addEventListener('input', () => {  
  values[index] = +slider.value  
  const valueEl = $(`#v${index}`)  
  if (valueEl) {  
    const numEl = valueEl.querySelector('.servo__number')  
    if (numEl) numEl.textContent = slider.value  
  }  
  
  updateTrackFill(index)  
  updatePacketPreview()  
  setServoAngle(index, values[index])  
  updateOledAnglesPreview()  
  
  scheduleServoSend()  
})
```

Формирование serial-пакета:

```
function buildPacket(): string {  
  return SERVO_CONFIG.map((s, i) => s.id + values[i]).join("") + ';' }  
}
```

Пример результата:

A100B120C60D90E90;

Передача данных через Web Serial API

Модуль src/serial.ts отвечает за транспортный уровень. Он открывает порт, создаёт reader/writer и передаёт данные между браузером и Arduino.

Отправка команды:

```
export async function send(data: string): Promise<void> {  
  if (!writer) {  
    throw new Error('Не подключено')  
  }  
  
  await writer.write(textEncoder.encode(data))  
}
```

Отправка servo-пакета из main.ts:

```
async function sendServoPacket(packet = buildPacket(), force = false) {
  if (!serial.isConnected()) return
  if (!force && packet === lastSentServoPacket) return

  try {
    await serial.send(packet)
    lastSentServoPacket = packet
    addLogEntry('tx', packet)
  } catch (err) {
    const msg = err instanceof Error ? err.message : 'Ошибка отправки'
    addLogEntry('err', msg)
  }
}
```

Параметр force используется для ручной кнопки Отправить: пользовательская отправка должна уходить даже если пакет совпадает с предыдущим.

Обработка пакета на Arduino

Arduino читает входящие символы до ;, после чего вызывает handleCommand.

Фрагмент чтения Serial:

```
void loop() {
  while (Serial.available() > 0) {
    char c = Serial.read();

    if (c == ';') {
      serialBuffer[serialBufferLength] = '\0';
      handleCommand(serialBuffer);
      serialBufferLength = 0;
      serialBuffer[0] = '\0';
      continue;
    }

    if (c == '\n' || c == '\r') continue;
    // накопление символов команды
  }
}
```

Разбор одного канала:

```
bool parseChannel(const char *command, char channel,
                 int minValue, int maxValue, int &value) {
  const char *cursor = strchr(command, channel);
  if (cursor == NULL) return false;

  cursor++;
  if (!isDigit(*cursor)) return false;
```

```

int parsed = 0;
while (isDigit(*cursor)) {
    parsed = parsed * 10 + (*cursor - '0');
    cursor++;
}

value = constrain(parsed, minValue, maxValue);
return true;
}

```

Разбор всего servo-пакета:

```

bool parseServoPacket(const char *command, int nextAngles[SERVO_COUNT]) {
    for (byte i = 0; i < SERVO_COUNT; i++) {
        if (!parseChannel(command, servoIds[i],
                          servoMin[i], servoMax[i], nextAngles[i])) {
            return false;
        }
    }

    return true;
}

```

Применение углов:

```

void applyServoAngles(const int nextAngles[SERVO_COUNT]) {
    setupServos();

    for (byte i = 0; i < SERVO_COUNT; i++) {
        servoAngles[i] = nextAngles[i];
        servos[i].write(servoAngles[i]);
    }

    if (oledMode == OLED_MODE_ANGLES) {
        showAngles();
    }
}

```

Сервоприводы подключаются лениво: setupServos() вызывается только при первой servo-команде. Это уменьшает вероятность рывка сразу после старта прошивки.

```

void setupServos() {
    if (servosAttached) return;

    for (byte i = 0; i < SERVO_COUNT; i++) {
        servos[i].attach(servoPins[i]);
    }

    servosAttached = true;
}

```

Связь с цифровым двойником Three.js

3D-модель используется как цифровой двойник роборуки. При движении слайдеров она обновляется сразу, независимо от того, подключена Arduino или нет.

Публичная функция установки угла:

```
export function setServoAngle(servoIndex: number, angle: number) {  
  switch (servoIndex) {  
    case 0:  
      targetAngles.base = angle  
      break  
    case 1:  
      targetAngles.shoulder = angle  
      break  
    case 2:  
      targetAngles.elbow = angle  
      break  
    case 3:  
      targetAngles.wrist = angle  
      break  
    case 4:  
      targetAngles.gripper = angle  
      break  
  }  
}
```

Применение углов к объектам модели:

```

if (parts.base) {
  parts.base.rotation.y = THREE.MathUtils.degToRad(targetAngles.base)
}
if (parts.shoulder) {
  parts.shoulder.rotation.x = THREE.MathUtils.degToRad(
    targetAngles.shoulder - 90
  )
}
if (parts.elbow) {
  parts.elbow.rotation.x = THREE.MathUtils.degToRad(targetAngles.elbow - 90)
}

```

Смещение -90 для плеча и локтя — это калибровка под нулевую позу экспортированной GLB-модели. В ходе работы этот mapping не изменялся, чтобы не сломать уже рабочее управление 3D-моделью.

Проверка на реальном стенде

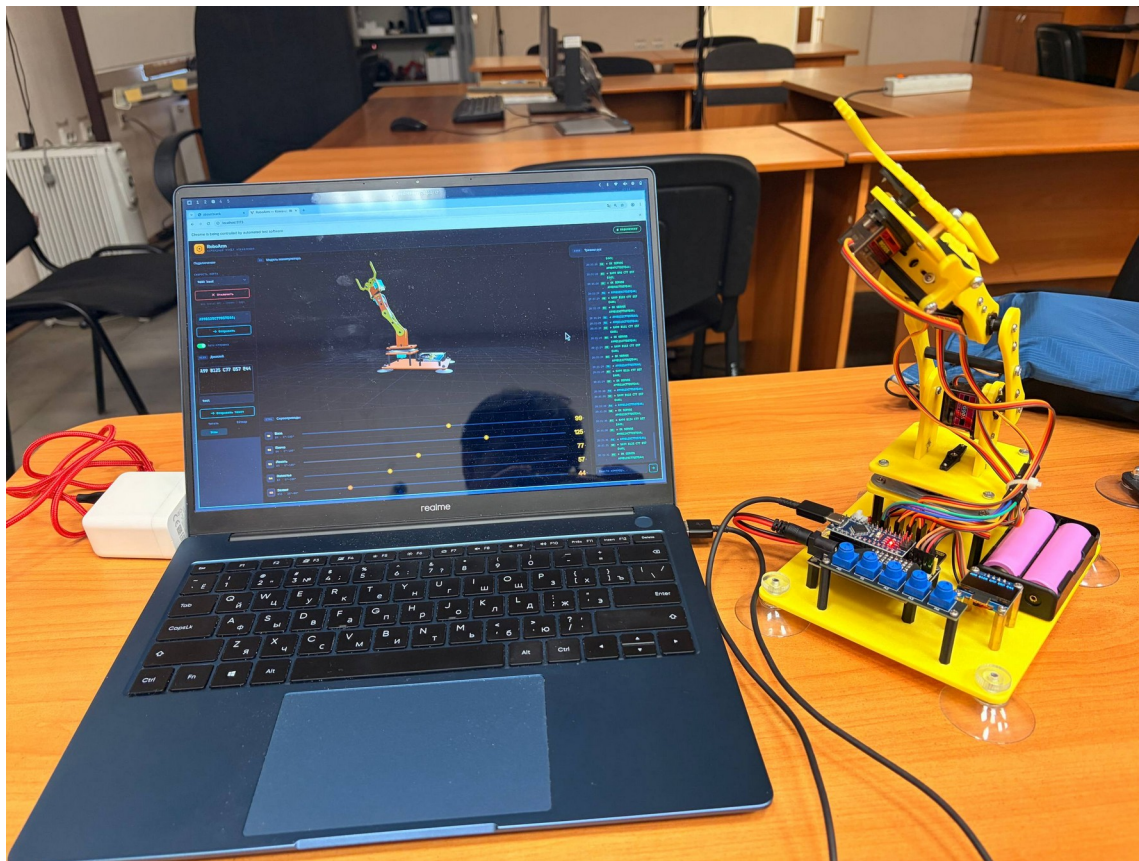


Рис.1 — подключение и проверка

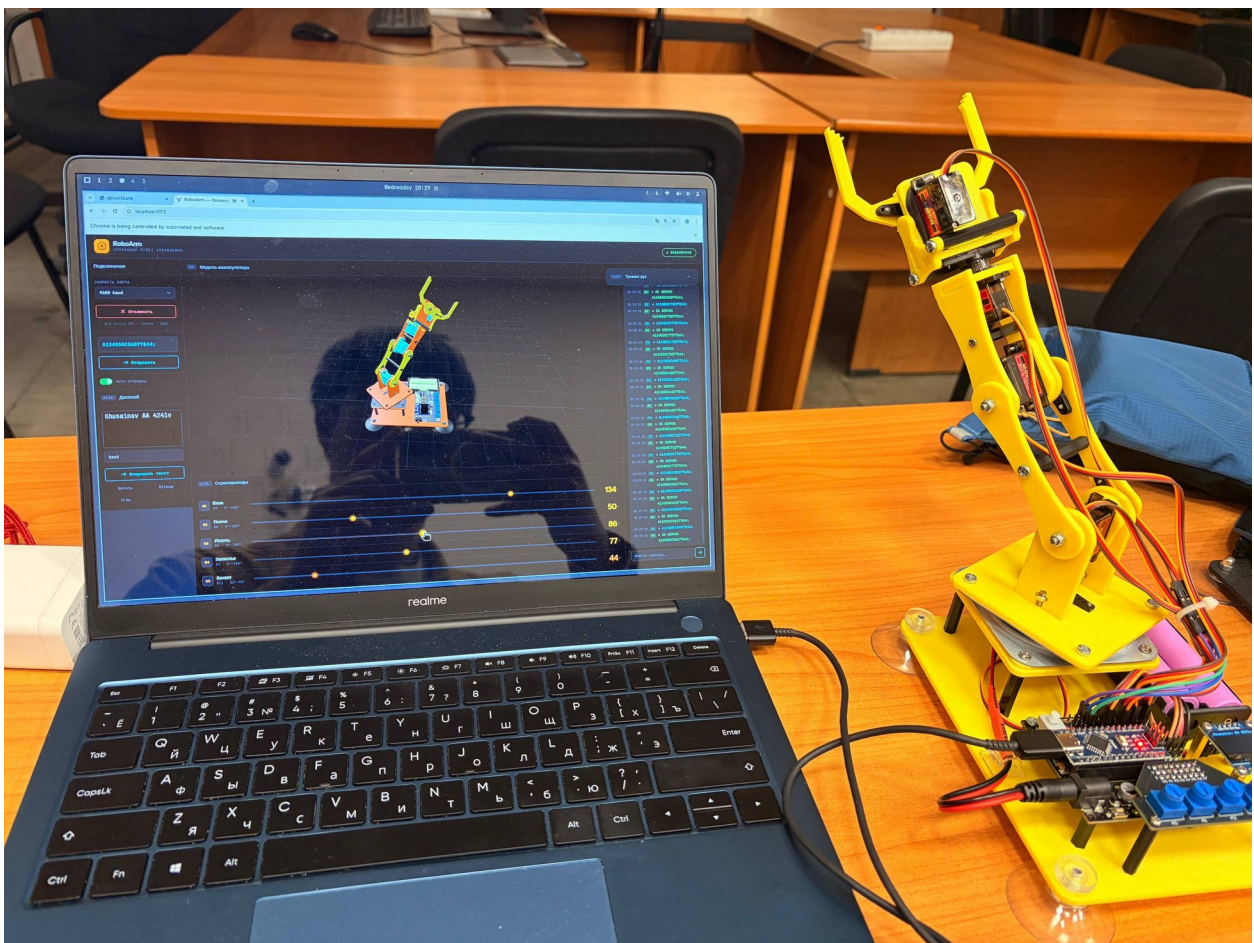


Рис.2 — подключение и проверка

Throttled debounce авто-отправки

Обычный HTML-слайдер генерирует много событий input во время движения. Если отправлять serial-пакет на каждое событие, можно создать слишком плотный поток команд. Для Arduino на 9600 baud это нежелательно: устройство будет постоянно получать команды, а лог интерфейса быстро заполнится.

В проекте реализован throttled debounce. Это не классический debounce, который ждёт полной остановки пользователя. Здесь используется другой подход:

- при каждом изменении слайдера сохраняется последний актуальный пакет;
- если таймер уже запущен, новый таймер не создаётся;
- через 80 мс отправляется последний сохранённый пакет;
- при непрерывном движении получается ограничение частоты отправки примерно до 12.5 пакетов в секунду;
- одинаковые пакеты не отправляются повторно.

Константа задержки:

```
const SERVO_SEND_DEBOUNCE_MS = 80
```

Переменные для авто-отправки:

```

let autoSend = false
let pendingServoPacket: string | null = null
let servoSendTimer: ReturnType<typeof setTimeout> | null = null
let lastSentServoPacket = "

```

Планирование отправки:

```

function scheduleServoSend() {
  if (!autoSend || !serial.isConnected()) return

```

```

  pendingServoPacket = buildPacket()
  if (servoSendTimer) return

```

```

  servoSendTimer = setTimeout(() => {
    const packet = pendingServoPacket
    pendingServoPacket = null
    servoSendTimer = null
    if (packet) sendServoPacket(packet)
  }, SERVO_SEND_DEBOUNCE_MS)
}

```

Защита от повторной отправки такого же пакета:

```

if (!force && packet === lastSentServoPacket) return

```

Ручная отправка работает иначе:

```

btnSend?.addEventListener('click', async () => {
  if (!serial.isConnected()) return

  cancelPendingServoSend()
  await sendServoPacket(buildPacket(), true)
})

```

Здесь force = true, поэтому пользовательская команда отправляется всегда.

Преимущество такой реализации: 3D-модель и UI остаются отзывчивыми и обновляются на каждое движение слайдера, но Arduino получает команды с ограниченной частотой.

Результаты проверки

Прошивка была скомпилирована для Arduino Nano и Arduino Uno.

Результат для Nano:

Sketch uses 19756 bytes (64%) of program storage space.
Global variables use 1017 bytes (49%) of dynamic memory.

Результат для Uno:

Sketch uses 19756 bytes (61%) of program storage space.
Global variables use 1017 bytes (49%) of dynamic memory.

Проверка через веб-интерфейс на тестовой Uno показала:

```
TX A100B120C60D90E90;  
RX %A100 B120 C60 D90 E90%;  
RX OK SERVOS A100B120C60D90E90;
```

Ответ %A100 B120 C60 D90 E90%; появился потому, что был включён режим OLED_ANGLES?. Это удобно для отладки: OLED-контур подтверждает актуальные углы, а строка OK SERVOS ... подтверждает успешное применение servo-пакета.

Также была проверена защита диапазона захвата:

```
A0B0C0D0E180;  
OK SERVOS A0B0C0D0E90;
```

Значение E180 было ограничено до E90, что подтверждает работу constrain.

Вывод

В работе реализовано управление роботом через веб-интерфейс и СОМ-порт. Пользователь меняет углы слайдерами, веб-приложение сразу обновляет цифровой двойник на Three.js и формирует serial-пакет для Arduino. Arduino принимает команду, проверяет наличие всех каналов, ограничивает значения безопасными диапазонами и вызывает Servo.write.

Особое внимание уделено авто-отправке. Throttled debounce позволяет не перегружать СОМ-порт и Arduino большим количеством команд при движении слайдеров. При этом интерфейс остаётся отзывчивым, а последняя актуальная позиция регулярно отправляется на устройство.